Question Bank — Week 1

Why this course · Python basics (Lecture 01, Lab 01) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **4**

Part C — Trace the code **7**

Part D — Find and fix the bug **7**

Part E — Write the code **8**

> ***Questions:*** `week01-questions.md`. *Read an answer only after you have committed to your own. For multiple choice, the explanation of why each wrong option is wrong is the part worth studying: the wrong options are the mistakes students actually make.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	b	M13	b	M19	c
M02	d	M08	a	M14	c	M20	a
M03	c	M09	c	M15	d	M21	d
M04	c	M10	b	M16	a	M22	b
M05	a	M11	d	M17	c	M23	c
M06	d	M12	a	M18	b	M24	d

W1-M01 — b. Turing-completeness is about *what can be computed*: the same set of problems. It says nothing about speed (a), is not the same claim as "only the syntax differs" (c) — languages differ in cost to write, read and run — and programs are certainly not portable between languages (d).

W1-M02 — d. *Dynamic*: types are checked at run time and a name can be rebound to a different type. *Strong*: Python refuses to mix incompatible types silently (see M03). "Static" (a, b) describes C, Java, Rust; "weak" (b, c) describes C's implicit conversions or JavaScript's `"3" + 5`.

W1-M03 — c. `TypeError: can only concatenate str (not "int") to str`. `"35"` (a) is JavaScript's answer — weak typing. `8` (b) would need an implicit conversion Python refuses to make. It is valid syntax, so not (d).

W1-M04 — c. Ownership and borrowing are checked at compile time. Manual `free` (a) is C/C++; a garbage collector (b) is Python, Java, Go; (d) describes no mainstream language.

W1-M05 — a. Ritchie, Bell Labs, early 1970s, grown out of Thompson's B, in order to write Unix (rewritten in C in 1973). Stroustrup (b) created C++; Knuth (c) created TeX; van Rossum (d) created Python.

W1-M06 — d. The ADT is the *contract*. Memory layout (a) is exactly what it leaves out — that belongs to the data structure. The language (b) and class hierarchy (c) are implementation choices.

W1-M07 — b. A generator expression inside `sum` states the result wanted. (a) and (d) are procedural — steps and state; (c) is functional — composing `map` and `filter`.

W1-M08 — a. TIOBE counts search-engine hits. It measures how much people write about a language, not quality (b); GitHub activity is Octoverse (c); self-reported usage is the Stack Overflow survey (d).

W1-M09 — c. Know-what is (a) and (d) — recall. Know-how is (b) — building it. Know-why is choosing and justifying by cost, which is what the exam weighs.

W1-M10 — b. Both thresholds, from the bylaws: ≥ 60% overall **and** ≥ 30% of the final. (c) is the common misreading — a strong coursework mark cannot rescue a final below 30%. And attendance below 75% bars you from the final entirely.

W1-M11 — d. `//` is *floor* division: it rounds **down**. −3.5 rounded down is −4. `-3` (a) is rounding towards zero, which is C's behaviour; `-3.5` (b) is what `/` gives.

W1-M12 — a. Python keeps `a == (a // b) * b + a % b`. Here `-7 // 3` is −3, and −3 × 3 + 2 = −7, so the remainder is 2. With a positive divisor, `%` is never negative in Python.

W1-M13 — b. `**` groups right to left: `2 ** (3 ** 2)` = 2^9 = 512. `64` (a) is `(2 ** 3) ** 2`.

W1-M14 — c. `/` always returns a `float`, even when the division is exact: `8 / 4` is `2.0`. Use `//` for an integer result.

W1-M15 — d. Indices 1, 2, 3 — up to but **not including** 4: `"yth"`. `"ytho"` (b) includes index 4.

W1-M16 — a. Step −2 from the end: indices 7, 5, 3, 1 → `a`, `u`, `s`, `a`. `"aruosnaM"` (b) is `[::-1]`; `"Mnor"` (c) is `[::2]`.

W1-M17 — c. `b = a` copies nothing: both names refer to the **same** list, so appending through `b` changes what `a` sees.

W1-M18 — b. `a[:]` makes a new list with equal contents: a different object (`is` is `False`) with the same values (`==` is `True`).

W1-M19 — c. `[[0] * 2] * 2` puts **two references to one inner list** into the outer list. Changing it through `grid[0]` shows through `grid[1]`. The correct construction is `[[0] * 2 for _ in range(2)]`.

W1-M20 — a. Python rounds exact halves to the nearest **even** number: `round(2.5)` is 2 and `round(3.5)` is 4 — total 6.

W1-M21 — d. A float is a binary fraction; 0.1 has no exact binary representation, so `0.1 + 0.2` is `0.30000000000000004`. Not a bug (a) — every language with IEEE floats does this. Compare floats with a tolerance.

W1-M22 — b. Strings are immutable. (c) is how you *should* do it — `s = "z" + s[1:]` — but item assignment itself is refused.

W1-M23 — c. A function with no `return` returns `None`. The `12` appears on the screen, but it is not *returned*. Tests see only what is returned.

W1-M24 — d. `int()` of a string accepts only an integer literal. Use `int(float("4.5"))` if truncation is really what you want.

W1-M25 — c. "A portable assembler; trust the programmer" is C's founding decision. A bounds check costs time on every access, so C leaves it out. It is not a bug (a). Python is not compiled to machine code (b); it checks because its designers preferred a loud failure to silent corruption.

W1-M26 — a. A Python `int` grows as large as memory allows, so the answer is exact. Wrapping round (b) is what a fixed-size C integer does. `OverflowError` (c) is raised by some *float* operations, not by `int` arithmetic, and (d) would need a float in the expression.

W1-M27 — d. `tests/` calls each function with arguments and checks what it returns, and a test cannot type at a keyboard. `input()` exists in Python 3 (c) and returns a **string** (b), never an `int`.

W1-M28 — b. `:03d` means an integer, at least 3 wide, padded with zeros: `007`. `:.2f` means two digits after the point: `3.14`. (a) ignores the format, and (d) has the wrong precision.

W1-M29 — a. `b = a` makes two names for one list. Then `a = a + [4]` builds a **new** list and rebinds only `a`. `b` still names the old list, so `b.append(5)` changes only that one. (b) and (c) assume that `+` changes the list in place; `append` does, `+` does not.

W1-M30 — d. The list holds references, as the ladder in Lecture 01 shows, so the objects themselves can be of any type. Nothing is converted (a), and Python lists declare no element type (b).

W1-M31 — b. Correctness first, because a fast wrong answer is worthless; then time, then space. (a) puts speed before correctness, which is exactly the habit the lecture warns against.

W1-M32 — a. Big-O describes growth and throws constant factors away. Binary search does more work per step, so at small n linear search can win: the **crossover point** that a measurement shows. The measurement is not wrong (b), and binary search is $O(\log n)$ at every size (c).

W1-M33 — c. $O(n^2)$: ten times the data means $10^2 = 100$ times the work, so about 200 seconds. (a) is what an $O(n)$ program would do.

W1-M34 — b. TIOBE counts search results, a proxy for how much people write about a language. The lecture keeps two uses for rankings, trends over years and "is it still alive?", so (d) goes too far, and neither (a) nor (c) follows from a search count.

W1-M35 — a. Splitting on an exact separator keeps the empty string between two commas that are next to each other. It is `split()` with **no** argument that splits on runs of whitespace and

drops empty pieces, which is where (b) comes from.

W1-M36 — d. Most decimal fractions are not exact in binary (`0.1 + 0.1 + 0.1 == 0.3` is `False`), and the errors add up over many sums. Store the smallest unit as an `int`, which is exact. Floats can be negative (b) and go far above a million (c).

W1-M37 — b. `find` returns `-1` when the substring is absent; `index` raises `ValueError` (`substring not found`). Neither returns `None` (d), and the error is a `ValueError`, not an `IndexError`.

W1-M38 — a. The course studies how structures work and what they cost. Calling the built-in that does it for you hides both, which is why the course re-implements them and, from Lecture 02, forbids `list`, `dict` and `set` as storage inside `dsa/`. The built-ins are usually **faster** than student code, because they run in C, so (b) is the wrong way round.

W1-M39 — c. Stroustrup's rule for C++ was that it should leave no gap below it. "Readability" is van Rossum's Python (a), "fewer features" is Go, by Pike, Thompson and Griesemer (b), and "trust the programmer" is Ritchie's C (d).

W1-M40 — d. In Python the indentation *is* the block structure, so there is no separate layout that could be bad while the code stays the same. Nothing re-indents your code (a, c), and the indentation is not ignored (b).

W1-M41 — a. `return a, b` returns **one** value, a tuple. Here `9 < 2` is false, so the second `return` gives `(2, 9)`, and `result[0]` is `2`. (c) takes the order of the arguments instead of the order returned.

W1-M42 — b. `grid[:]` is a **shallow** copy: a new outer list holding the same inner lists. `copy[0][0] = 9` changes an inner list that both share, so `grid` sees it. `copy[1] = [7, 7]` rebinds a slot of the **new** outer list only, so `grid` does not see it, which rules out (c). (a) assumes a deep copy.

Part B — Short answer and essay

Model answers. The **marking points** are what an examiner looks for; each is roughly one mark.

W1-E1 *(4)*

- **True part:** general-purpose languages are Turing-complete, so they can all compute the same set of functions.
- **What is wrong:** equivalence of *what* can be computed says nothing about the **cost** — to write, to read and maintain, to run — nor about which mistakes the language lets survive into production.
- A language is a **set of design decisions** with authors and reasons (language → design → philosophy).
- **Example**, any one well explained: C trusts the programmer and does no bounds checking, so `a[10]` on a 5-element array silently corrupts memory, while Python raises `IndexError`;

or Python is strongly typed (`"3" + 5` is an error) while JavaScript is weak (`"35"`); or Rust's ownership versus C's manual `free`.

W1-E2 *(4)*

- **Static vs dynamic** is *when* types are checked — at compile time, or at run time.
- **Strong vs weak** is *how strictly* types are kept apart — refusing, or silently converting.
- Python is **dynamic and strong**; JavaScript is **dynamic and weak**.
- `"3" + 5`: Python raises `TypeError`; JavaScript returns `"35"`. Neither is a bug — they are different decisions about loud failure versus convenient conversion.

W1-E3 *(3)*

- **Manual** (C, C++): the programmer allocates and frees — fast, and the source of many security bugs. **Garbage collected** (Python, Java, Go): the runtime frees unreachable objects — safe, less control. **Ownership** (Rust): the compiler proves each allocation is freed exactly once — safe and fast, harder to satisfy.
- Python uses **garbage collection** (reference counting plus a cycle collector).
- Consequence: removing a node from a linked list is just "stop referring to it" — no `free`, no visible pointer — so the structure is invisible unless you use the debugger to watch references change.

W1-E4 *(4)*

- **ADT**: the contract — operations, their meaning, their cost — with nothing about storage.
- **Data structure**: a concrete arrangement of data in memory that fulfils the contract.
- **Stack ADT**: `push` adds, `pop` removes the most recent, `peek` looks; all O(1).
- **Two structures**: a dynamic array with the top at the end; a linked list with the top at the head. Both honest stacks; they differ in memory overhead, cache behaviour and whether one operation can occasionally be slow. Choosing between them is the engineering.

W1-E5 *(3)*

- **For**, any two: the field (research, data, AI/ML) runs on Python; it binds to fast C/C++/CUDA underneath; it reads close to pseudocode, so the algorithm survives onto the page; students meet it again in every program.
- **Against:** Python already has the structures as built-ins — `list`, `dict`, `deque`, `sorted`, `heapq` — which hides exactly what the course studies.
- **Remedy:** the course reimplements them, and (from Lecture 02) structures may store data only in the course `Array`, in nodes, or in structures students built — never in `list`, `dict` or `set`.

W1-E6 *(3)*

- `a = [1, 2, 3]` creates a list object and binds the name `a` to it; `b = a` binds a **second name to the same object** — no copy.
- Diagram: two names, `a` and `b`, with arrows to one list object.
- Consequence: `b.append(4)` changes what `a` shows. To get an independent list, copy it: `a[:]`, `a.copy()` or `list(a)` — and that copy is **shallow**.

W1-E7 *(4)*

- **Why measure:** a claim about cost is a hypothesis. Timing the code over growing n and plotting it against reference curves checks the claim instead of arguing about it (`viz.complexity.measure` and `plot_growth`).
- **What it shows that Big-O does not:** the **constant factors**, and so the **crossover point**. Linear search can beat binary search at small n, because binary search does more work per step, even though it wins at scale.
- **How it can mislead:** one run is noisy, a small n hides the growth, and one machine is not every machine.
- **Avoiding it:** time many sizes over a wide range, repeat the runs and take the best or the average, and judge the **shape** (the slope on a log–log plot), not one number.

W1-E8 *(3)*

- `def f(x): print(x * 2)` shows the value on the screen and returns `None`; `def g(x): return x * 2` hands the value back to the caller.
- A test sees only what is **returned**: `assert f(3) == 6` fails, because `f(3)` is `None`, although "6" appeared on the screen.
- Exercises take **parameters** and **return** results because a test can call them with any input and check the output automatically. A function that calls `input()` waits for a keyboard that a test does not have, and `print()` gives the test nothing to check.

W1-E9 *(4)*

- The real questions: best **for what task** (firmware, a web service, data analysis…), **for whom** (one learner, or a large team for ten years), **by which measure** (speed to write, speed to run, safety, hiring), and **when** (this project, or one maintained until 2035).
- Change any one of them and the answer changes, so "best" alone has no answer.
- **Ecosystem**, any three: libraries, tooling (debugger, profiler, package manager, test runner), community (answers already written), interoperability (calling fast code written in another language), longevity.
- **Why it beats syntax:** you stop noticing syntax after two weeks, but you work inside the ecosystem for years. Whether someone has already solved your problem, well, matters more than how a loop is spelt.

W1-E10 *(4)*

- **C `int`** has a fixed size (usually 32 bits) and silently **wraps round** when it overflows: $2^{31} - 1$ plus 1 becomes -2^{31}.
- **Python `int`** grows as large as memory allows, so `2 ** 100` is exact. The cost is that arithmetic on huge integers is **not** $O(1)$: it takes longer as the number of digits grows.
- A **`float`** is a 64-bit binary fraction. 0.1, 0.2 and 0.3 have no exact binary form, so `0.1 + 0.2` is `0.30000000000000004`. This is not a bug; C, Java and JavaScript do the same.
- **Rules:** never compare floats with `==` after arithmetic (round them, or test `abs(x - y) < 1e-9`), and never store money as a float (store piasters as an `int`).

Part C — Trace the code

W1-T1

```
3 2 -4 3
(5, 3) 1024 3.5
```

`-17 // 5` is −3.4 rounded **down**, −4; and −4 × 5 + 3 = −17, so `-17 % 5` is 3.

W1-T2

```
D s Stru DATA
['Data', 'Structures'] 15 3
```

`s[5:9]` is indices 5, 6, 7, 8 — `S t r u`. `split()` with no argument splits on runs of whitespace. The three `t`s: one in *Data*, two in *Structures*.

W1-T3

```
[99, 2, 3, 4] [99, 2, 3, 4] [1, 2, 3, 5]
[99, 2, 3, 4, 6]
```

`a` and `b` are one list, so `append` and `b[0] = 99` both show in both. `c` was a copy taken *before* those changes; `c + [5]` builds a new list. `a += [6]` extends the list **in place**, so `b` sees it — unlike `a = a + [6]`, which would have made a new list for `a` only.

W1-T4

```
1 6
```

1000 → 333 → 111 → 37 → 12 → 4 → 1: six divisions by 3. Dividing by 3 until 1 takes about $\log_3 n$ steps — the Week 2 idea of logarithmic cost.

W1-T5

```
  Omar|87.5|   87.46|004
```

`>6` right-aligns in 6 characters; `.1f` rounds to one decimal; `8.2f` is 8 characters wide with two decimals; `03d` pads the integer to 3 digits with zeros.

W1-T6

```
[0, 1, 1, 2, 3, 5, 8, 13]
[5, 8, 13] [0, 2, 8]
```

`[::3]` takes indices 0, 3, 6.

Part D — Find and fix the bug

W1-B1. `split(" ")` splits on **each single space**, so two spaces in a row, or a leading or trailing space, produce empty strings `""`, and `""[0]` raises `IndexError`. Input that fails: `"Ada`

`Lovelace"` or `"  Ada"`. **Fix:** use `split()` with no argument, which splits on runs of whitespace and drops empty pieces.

W1-B2. `i` is never incremented, so the loop never ends for any non-empty list — the program hangs (Ctrl+C to stop it). **Fix:** add `i += 1` inside the loop — or, better, `for v in values: total += v`, which cannot forget.

W1-B3. `result = values` does **not** copy; append then changes the caller's list, which the specification forbids. Test: after `with_item(original, 9)`, `original` has changed. **Fix:** `result = values[:]` (or `values.copy()`, or simply `return values + [item]`).

Part E — Write the code

Worked solutions. Yours may differ and still be correct — the tests decide.

W1-C1

```
def count_vowels(text):
    count = 0
    for ch in text.lower():
        if ch in "aeiou":
            count += 1
    return count
```

W1-C2 — one pass: keep the largest and the second largest seen so far.

```
def second_largest(values):
    largest = second = None
    for v in values:
        if largest is None or v > largest:
            if largest is not None:
                second = largest          # the old maximum is now second
            largest = v
        elif v != largest and (second is None or v > second):
            second = v
    if second is None:
        raise ValueError("need at least two distinct values")
    return second
```

The `v != largest` check is what makes `[9, 9, 7]` give 7, not 9. Sorting would work too but is O(n log n); this is O(n).

W1-C3 — find where each run ends.

```
def compress(text):
    result = ""
    i = 0
    while i < len(text):
```

```
        j = i
        while j < len(text) and text[j] == text[i]:
            j += 1                          # j stops at the end of the run
        result += text[i] + str(j - i)
        i = j
    return result
```

Each character is looked at a constant number of times, so this is O(n) steps (ignoring the cost of building the string).

W1-C4

```
def is_anagram(first, second):
    a = sorted(first.replace(" ", "").lower())
    b = sorted(second.replace(" ", "").lower())
    return a == b
```

O(n log n) because of sorting. Counting letters in a dictionary (Lab 03) makes it O(n).